

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών

6η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614

Φίλιππος Γιαννακόπουλος 03121629

Ομάδα 17

Ζήτημα 6.1

Το Ζήτημα 6.1 βρίσκεται στο αρχείο erg6.1.c.

Κάνουμε χρήση της scan_keypad και keypad_to_ascii για να δούμε ποιο κουμπί είναι ενεργοποιημένο ανά πάσα στιγμή.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//Fsc1=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDR0;
}

// Read one byte from twi device (followed by a stop condition)
```

```

unsigned char twi_readNak() {
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // Wait till address transmission is completed and ACK/NACK has been received
        while (!(TWCR0 & (1<<TWINT)));
    }
}

```

```

// Check TWI Status
twi_status = TW_STATUS & 0xF8;
if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
    // Device is busy, send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));

    continue;
}

break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCRO = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCRO & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));
}

// PCA9555 functions
void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
}

```

```

    ret_val = twi_readNak();
    twi_stop();

    return ret_val;
}

// Keypad functions
const uint8_t rows[] = {0b11111110, 0b11111101, 0b11111011, 0b11110111};
const char characters[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
uint16_t pressed_keys = 0x0000;

uint8_t scan_row(uint8_t row) {
    PCA9555_0_write(REG_OUTPUT_1, rows[row]);
    uint8_t input = PCA9555_0_read(REG_INPUT_1);
    input = ~input;
    return input>>4;
}

uint16_t scan_keypad() {
    uint16_t output = 0x0000;
    uint8_t temp;
    for (uint8_t i = 0; i < 4; i++) {
        output = output<<4;
        temp = scan_row(i) & 0x0F;
        output |= temp;
    }

    return output;
}

void scan_keypad_rising_edge() {
    uint16_t pressed_keys_tempo = scan_keypad();
    _delay_ms(10);
    pressed_keys_tempo &= scan_keypad();

    pressed_keys = pressed_keys_tempo & ~(pressed_keys & pressed_keys_tempo);
}

char keypad_to_ascii(uint16_t keys) {
    uint16_t test = 1;
    for (uint8_t i = 0; i < 16; i++) {
        if (keys & test) {
            return characters[i];
        }
        test = test<<1;
    }
    return '\0';
}

int main(void) {
    twi_init();

    // Set PORTB to an input
    DDRB = 0xFF;
    PORTB = 0x00;

    // Set EXT_PORT0 as output
    PCA9555_0_write(REG_CONFIGURATION_1, 0b11110000);

    while (1) {
        pressed_keys = scan_keypad();
    }
}

```

```

char ascii = keypad_to_ascii(pressed_keys);

switch (ascii) {
    case 'A':
        PORTB = 1;
        break;
    case '8':
        PORTB = 1<<1;
        break;
    case '6':
        PORTB = 1<<2;
        break;
    case '*':
        PORTB = 1<<3;
        break;
    default:
        PORTB = 0x00;
}
}
}

```

Ζήτημα 6.2

Το Ζήτημα 6.2 βρίσκεται στο αρχείο erg6.2.c.

Χρησιμοποιούμε την επικοινωνία με την LCD μέσω του PORTD.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//Fsc1=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCRR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCRR0 & (1<<TWINT)));

    return TWDRR0;
}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
```

```

    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // Wait till address transmission is completed and ACK/NACK has been received
        while (!(TWCR0 & (1<<TWINT)));
    }
}

```



```

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
        // Device is busy, send STOP condition
        TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

        // Wait till STOP condition is executed and bus released
        while (!(TWCR0 & (1<<TWSTO)));

        continue;
    }

    break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCR0 & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCR0 & (1<<TWSTO)));
}

// PCA9555 functions
void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
}

```

```

    twi_stop();

    return ret_val;
}

// Keypad functions
const uint8_t rows[] = {0b11111110, 0b11111101, 0b11111011, 0b11110111};
const char characters[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
uint16_t pressed_keys = 0x0000;

uint8_t scan_row(uint8_t row) {
    PCA9555_0_write(REG_OUTPUT_1, rows[row]);
    uint8_t input = PCA9555_0_read(REG_INPUT_1);
    input = ~input;
    return input>>4;
}

uint16_t scan_keypad() {
    uint16_t output = 0x0000;
    uint8_t temp;
    for (uint8_t i = 0; i < 4; i++) {
        output = output<<4;
        temp = scan_row(i) & 0x0F;
        output |= temp;
    }

    return output;
}

void scan_keypad_rising_edge() {
    uint16_t pressed_keys_tempo = scan_keypad();
    _delay_ms(10);
    pressed_keys_tempo &= scan_keypad();

    pressed_keys = pressed_keys_tempo & ~(pressed_keys & pressed_keys_tempo);
}

char keypad_to_ascii(uint16_t keys) {
    uint16_t test = 1;
    for (uint8_t i = 0; i < 16; i++) {
        if (keys & test) {
            return characters[i];
        }
        test = test<<1;
    }
    return '\0';
}

// LCD Commands

void write2(unsigned char input) {
    unsigned char prev = PIND;

    PORTD = (input & 0xF0) | (prev & 0x0F);

    PORTD |= (1<<3);
    PORTD &= 0b11110111;

    PORTD = ((input & 0x0F) << 4) | (prev & 0x0F);

    PORTD |= (1<<3);
}

```

```

    PORTD &= 0b11110111;
}

void lcd_data(unsigned char input) {
    PORTD |= (1<<2);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    PORTD &= 0b11110111;
    write2(input);
    _delay_us(250);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}

void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x30;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    // Switch to 8bit mode
    PORTD = 0x20;
    PORTD |= (1<<3);
    PORTD &= 0b11110111;
    _delay_us(250);

    lcd_command(0x28);

    lcd_command(0x0c);

    lcd_clear();

    lcd_command(0x06);
}

int main(void) {
    DDRD = 0xFF;

    twi_init();
    lcd_init();
}

```

```

// Set EXT_PORT0 as output
PCA9555_0_write(REG_CONFIGURATION_1, 0b11110000);

char ascii;

while (1) {
    do {
        scan_keypad_rising_edge();
        ascii = keypad_to_ascii(pressed_keys);
    } while (ascii == '\0');

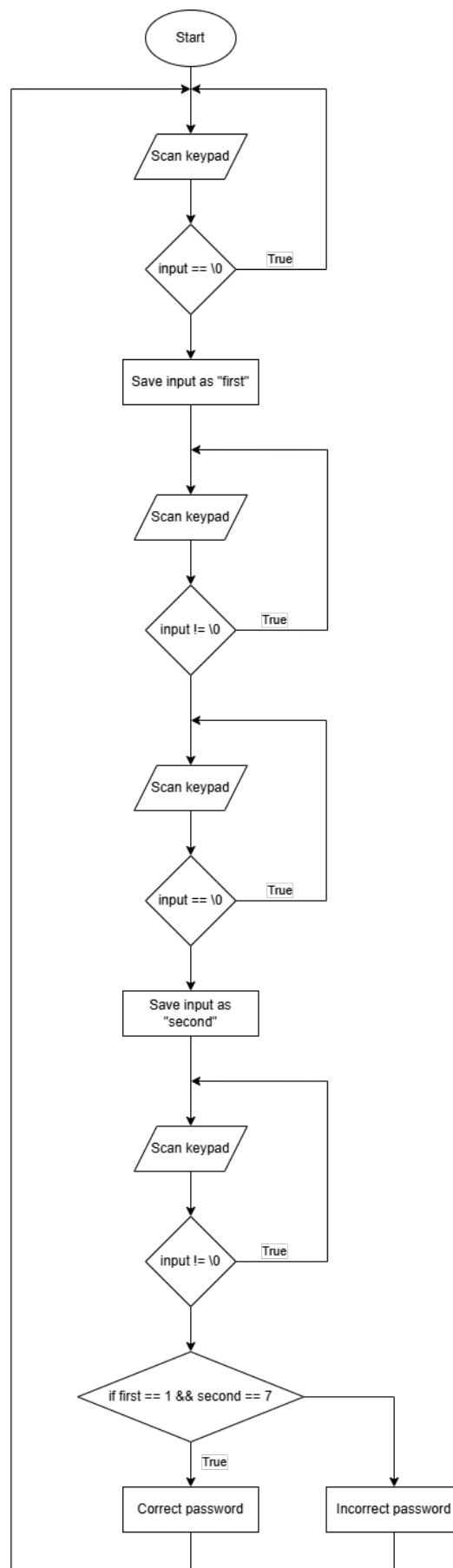
    lcd_clear();
    lcd_data(ascii);
}
}

```

Ζήτημα 6.3

Το Ζήτημα 6.3 βρίσκεται στο αρχείο erg6.3.c.

Διάγραμμα ροής:



```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//FscL=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDR0;
}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDR0;
}

```

```

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCRO = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCRO & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDR0 = address;
    TWCRO = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCRO & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCRO = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCRO & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDR0 = address;
        TWCRO = (1<<TWINT) | (1<<TWEN);

        // Wait till address transmission is completed and ACK/NACK has been received
        while (!(TWCRO & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
            // Device is busy, send STOP condition
            TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);
        }
    }
}

```

```

        // Wait till STOP condition is executed and bus released
        while (!(TWCR0 & (1<<TWSTO)));

        continue;
    }

    break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCR0 & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCR0 & (1<<TWSTO)));
}

// PCA9555 functions
void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();

    return ret_val;
}

```



```

// Keypad functions
const uint8_t rows[] = {0b11111110, 0b11111101, 0b11111011, 0b11110111};
const char characters[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
uint16_t pressed_keys = 0x0000;

uint8_t scan_row(uint8_t row) {
    PCA9555_0_write(REG_OUTPUT_1, rows[row]);
    uint8_t input = PCA9555_0_read(REG_INPUT_1);
    input = ~input;
    return input>>4;
}

uint16_t scan_keypad() {
    uint16_t output = 0x0000;
    uint8_t temp;
    for (uint8_t i = 0; i < 4; i++) {
        output = output<<4;
        temp = scan_row(i) & 0x0F;
        output |= temp;
    }

    return output;
}

void scan_keypad_rising_edge() {
    uint16_t pressed_keys_tempo = scan_keypad();
    _delay_ms(10);
    pressed_keys_tempo &= scan_keypad();

    pressed_keys = pressed_keys_tempo & (~pressed_keys);
}

char keypad_to_ascii(uint16_t keys) {
    uint16_t test = 1;
    for (uint8_t i = 0; i < 16; i++) {
        if (keys & test) {
            return characters[i];
        }
        test = test<<1;
    }
    return '\0';
}

int main(void) {
    DDRB = 0xFF;
    PORTB = 0x00;

    twi_init();

    // Set EXT_PORT0 as output
    PCA9555_0_write(REG_CONFIGURATION_1, 0b11110000);

    char ascii;
    char first;
    char second;

    while (1) {
        do {
            ascii = keypad_to_ascii(scan_keypad());
        } while (ascii == '\0');
    }
}

```

```

first = ascii;

while(keypad_to_ascii(scan_keypad()) != '\0');

do {
    ascii = keypad_to_ascii(scan_keypad());
} while (ascii == '\0');

second = ascii;

while(keypad_to_ascii(scan_keypad()) != '\0');

if (first == '1' && second == '7') {
    PORTB = 0b00111111;
    _delay_ms(5000);
    PORTB = 0x00;
} else {
    for (unsigned char i = 0; i < 5; i++) {
        PORTB = 0b00111111;
        _delay_ms(500);
        PORTB = 0x00;
        _delay_ms(500);
    }
}
}
}
}

```